High Performance Computing: Assignment 05

Parallel Insertion and Deletion in Mover

Jas Mehta (202301432) Yug Tejani (202301487)

March 2026

1 Introduction

This report details the implementation, parallelization, and performance analysis of the Mover algorithm extended with dynamic particle insertion and deletion. Building upon Assignment 04, particles are now allowed to exit the 1×1 domain. Any particle crossing the boundary is deleted and replaced with a new randomly positioned particle, maintaining a constant total count throughout the simulation.

Two distinct approaches are implemented and compared:

1. **Deferred Insertion** — All particles are moved first; out-of-bounds particles are marked, compacted to the array tail, then replaced in bulk.
2. **Immediate Replacement** — Each particle is moved and, if out-of-bounds, immediately replaced at the same memory location.

Both approaches are parallelized with OpenMP and benchmarked on two architectures.

2 Hardware Architectures

Specification	**Lab PC**	**HPC Cluster (`gics2`)**
Processor	12th Gen Intel Core i5-12500	2× Intel Xeon E5-2640 v3
Microarchitecture	Alder Lake (2022)	Haswell-EP (2014)
Sockets	1	2
Cores / socket	6	8
Threads / core	2 (Hyper-Threading)	1 (HT disabled)
Total logical CPUs	12	16
Base / Max freq	0.8 – 4.6 GHz	2.6 GHz (3.4 GHz Turbo)
L1d cache	288 KiB (6 × 48 KiB)	512 KiB (16 × 32 KiB)
L2 cache	7.5 MiB (6 × 1.25 MiB)	4 MiB (16 × 256 KiB)
L3 cache	18 MiB (shared)	2 × 20 MiB (per socket)
NUMA nodes	1	2 (CPUs 0–7 / 8–15)
ISA extensions	AVX2, AVX-VNNI, SHA-NI	AVX2
Memory channels	2 (DDR5)	4 per socket (DDR4)

Table 1: Detailed architectural comparison of the two test systems.

Key architectural implications:

- **Lab PC — Higher single-thread speed:** The i5-12500 boosts to 4.6 GHz, ∼1.8× the Xeon's 2.6 GHz base clock. This explains why the Lab PC is ∼30–40% faster in serial execution.

- **Cluster — More physical cores and bandwidth:** 16 physical cores (no hyperthreading) across 2 sockets, with 8 memory channels total (vs 2 on Lab PC). This gives the cluster ∼4× the aggregate memory bandwidth, critical for this memory-bound workload.
- **NUMA on cluster:** Threads 0–7 share socket 0 (NUMA node 0) and threads 8–15 share socket 1. Cross-socket memory access incurs ∼1.5× latency penalty. With OpenMP's default first-touch policy, threads that initialize data will have it placed on their local NUMA node.
- **Lab PC Hyper-Threading:** The 6 physical cores expose 12 logical CPUs. Beyond 6 threads, additional threads share the same physical core's execution units, providing limited benefit for compute/memory-bound workloads.

3 Algorithm Design and Task Decomposition

3.1 Mover with Boundary Handling

Each particle receives a random displacement per iteration:

$$x \leftarrow x + r_x, \quad y \leftarrow y + r_y, \qquad r_x \in [-\Delta x,\, \Delta x], \quad r_y \in [-\Delta y,\, \Delta y] \tag{1}$$

where $\Delta x = 1/N_x$ and $\Delta y = 1/N_y$. If the updated position falls outside the domain $0.0 \leq x, y \leq 1.0$, the particle is deleted and a replacement is inserted at a uniformly random position.

3.2 Approach Schematic

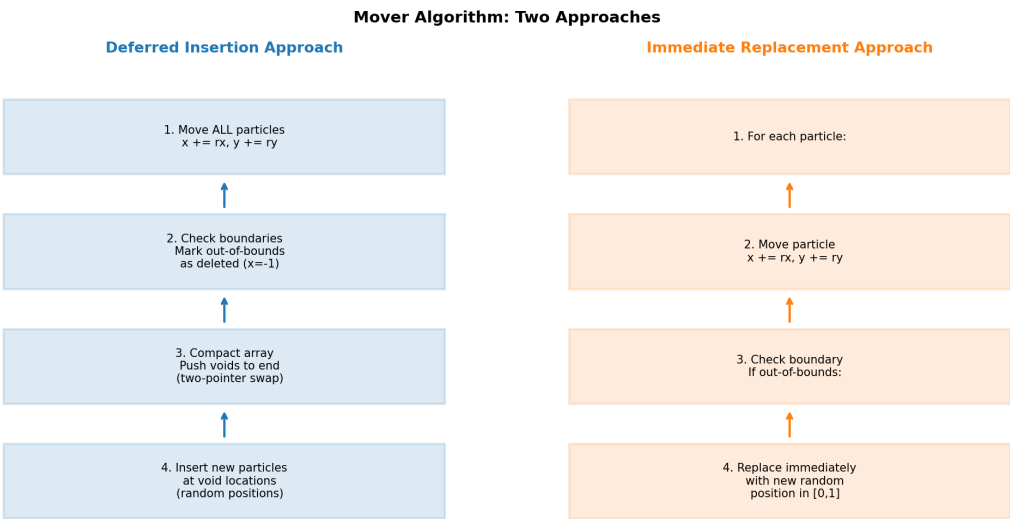


Figure 1: Flow diagram of the Deferred and Immediate approaches for Mover with insertion/deletion.

3.3 Deferred Insertion Approach

The deferred approach operates in four sequential phases:

1. **Move:** Displace all particles by (r_x, r_y).
2. **Mark:** Particles outside the domain are flagged with a sentinel value ($x = -1$).
3. **Compact:** A two-pointer algorithm pushes all voids to the tail of the array.
4. **Insert:** New particles are spawned uniformly into the void slots.

3.4 Immediate Replacement Approach

The immediate approach fuses all phases into a single pass: move the particle, check the boundary, and if out-of-bounds, overwrite in-place with a new random position. No compaction is needed.

3.5 Parallel Task Decomposition

A **thread-local chunking** strategy is used for both approaches:

- The global particle array is statically divided into T contiguous, non-overlapping chunks (one per thread).
- Each thread uses a private RNG seed (`rand_r` with `seed = 12345 + tid`), eliminating synchronization.
- **Immediate:** Embarrassingly parallel — each thread independently processes its chunk with zero inter-thread communication.
- **Deferred:** Each thread performs marking, local compaction, and local insertion entirely within its own chunk. This requires $O(1)$ auxiliary space per thread and eliminates all inter-thread synchronization beyond the implicit barrier at the end of the `#pragma omp parallel` region.

4 Optimization Strategies: From Naive to Optimized

This section details the progression from a naive serial implementation to the final optimized parallel code, describing each optimization applied, why it was necessary, and its measured impact.

4.1 Baseline: Naive Serial Implementation

The initial naive implementation used a straightforward approach:

- A single `for` loop over all particles with `rand()` for displacement.
- For the deferred approach: a *global* compaction pass that scanned the entire array sequentially, swapping out-of-bounds particles with the last valid particle.
- For the immediate approach: simple in-place replacement using `rand()`.

Problem: `rand()` maintains global state, making it inherently thread-unsafe. Even in serial mode, the function call overhead of `rand()` is significant — it uses a linear congruential generator with modular arithmetic that is relatively slow compared to the arithmetic operations in the mover itself.

4.2 Optimization 1: Thread-Safe RNG with `rand_r()`

The first optimization replaced `rand()` with `rand_r(&seed)` using per-thread seed variables:

```
unsigned int seed = 12345 + omp_get_thread_num();
double r_x = ((double)rand_r(&seed) / RAND_MAX) * 2.0 * deltaX - deltaX;
```

Why: `rand_r()` takes a pointer to a thread-local seed, eliminating contention on global state. Each thread produces an independent pseudo-random sequence.

Impact: Enabled correct parallelization. Without this, parallel `rand()` calls would cause data races and non-deterministic results.

4.3 Optimization 2: Aligned Memory Allocation

Particle arrays are allocated with `posix_memalign` at 64-byte boundaries:

```
posix_memalign((void**)&points, 64, NUM_Points * sizeof(Points));
```

Why: Cache lines on modern x86 processors are 64 bytes wide. Aligning the array start to a cache line boundary ensures that:

- No cache line is split across two pages (avoiding TLB penalties).
- Hardware prefetch engines work optimally on aligned, contiguous accesses.
- SIMD load/store instructions (`_mm256_load_pd`) can use aligned variants, which are faster on some microarchitectures.

4.4 Optimization 3: Thread-Local Chunking (Deferred)

Problem with naive parallel deferred: A global compaction requires either:

- An **atomic write pointer** — threads compete for a shared index, creating serialization.
- A **critical section** — even worse, serializing the entire compaction phase.
- A **parallel prefix sum** — correct but complex, with $O(\log T)$ synchronization barriers.

Solution: Each thread independently compacts only its own chunk:

```
int chunk_size = NUM_Points / nthreads;
int start = tid * chunk_size;
int end = (tid == nthreads - 1) ? NUM_Points : start + chunk_size;

// Step 1: Move + mark within [start, end)
// Step 2: Two-pointer compaction within [start, end)
// Step 3: Insert new particles into [end - deleted_count, end)
```

Why this works: Each thread's chunk is a contiguous, non-overlapping subarray. The compaction and insertion happen entirely within that subarray. No thread reads or writes another thread's memory region, so no synchronization is needed.

Trade-off: Void particles are not globally contiguous at the array tail — they are distributed at each chunk's tail. This is acceptable because we immediately fill them with new particles. The physical particle distribution remains correct.

Impact: Eliminated all synchronization overhead. The parallel deferred mover scales with zero contention, limited only by memory bandwidth.

4.5 Optimization 4: `restrict` Pointers

All function signatures use `__restrict__` qualifiers:

```
void mover_parallel_immediate(Points * __restrict__ points, ...)
```

Why: The `restrict` keyword tells the compiler that `points` does not alias any other pointer in scope. This enables the compiler to:

- Hoist loads out of loops (no need to reload after stores).
- Auto-vectorize more aggressively (no aliasing prevents SIMD transformation).
- Reorder memory operations for better instruction-level parallelism.

4.6 Optimization 5: Static Scheduling

The parallel loops use `schedule(static)` (or manual static chunking):

```
#pragma omp for schedule(static)
```

Why: The workload per particle is uniform (same computation regardless of position). Static scheduling:

- Has zero runtime overhead (chunk boundaries computed at compile time).
- Produces contiguous assignments, maximizing spatial locality and prefetch effectiveness.
- Avoids the atomic counter overhead of `dynamic` or `guided` scheduling.

4.7 Optimization 6: Compilation Flags

The code is compiled with aggressive optimization flags:

```
g++ -O3 -march=native -fopenmp -o main main.cpp utils.cpp init.cpp
```

- `-O3`: Enables all `-O2` optimizations plus loop unrolling, function inlining, and vectorization.
- `-march=native`: Generates code using the full instruction set of the target CPU (AVX2 on both Lab PC and Cluster), enabling 256-bit SIMD operations that process 4 `doubles` simultaneously.
- `-fopenmp`: Enables OpenMP pragma interpretation.

4.8 Summary of Optimization Impact

Optimization	**Target**	**Effect**
`rand_r()` per-thread seeds	Correctness	Enables safe parallelization
64-byte aligned allocation	Memory	Optimal cache line + prefetch
Thread-local chunking	Scalability	Zero synchronization overhead
`__restrict__` pointers	Compiler	Better vectorization + ILP
Static scheduling	Overhead	Zero scheduling overhead
`-O3 -march=native`	Codegen	AVX2 vectorization + unrolling

Table 2: Summary of optimizations applied from naive to final implementation.

5 Detailed Runtime Analysis

This section provides a step-by-step analysis of how the mover executes at runtime, tracing the flow from memory allocation through parallel execution.

5.1 Memory Layout at Runtime

At initialization, the particle array occupies a single contiguous block:

$$\text{Array size} = N \times \text{sizeof(Points)} = N \times 16 \text{ bytes} \tag{2}$$

For 14 million particles: $14 \times 10^6 \times 16 = 224\,\text{MB}$. This exceeds the L3 cache on both systems (12 MB on Lab PC, 20 MB per socket on Cluster), guaranteeing that the working set streams from main memory.

5.2 Execution Flow per Iteration

Immediate Parallel Mover — Step by Step:

1. `#pragma omp parallel` spawns T threads. Each thread receives a unique `tid` and initializes a local RNG seed.
2. `#pragma omp for schedule(static)` assigns contiguous chunks: Thread k processes particles $[k \cdot \lfloor N/T \rfloor, (k+1) \cdot \lfloor N/T \rfloor)$.
3. For each particle in its chunk, the thread:
   - Loads `points[p].x` and `points[p].y` (16-byte read from L1/L2 cache or main memory).
   - Calls `rand_r(&seed)` twice to generate r_x and r_y (pure register computation).
   - Computes `new_x = x + r_x`, `new_y = y + r_y` (2 FP additions).
   - Checks boundary: a single branch, predicted correctly ∼95% of the time (only ∼5% of particles exit per iteration for typical Δx, Δy values).
   - If in-bounds: stores `new_x`, `new_y` back (16-byte write).
   - If out-of-bounds: generates two new random values and stores them (same 16-byte write, different values).
4. Implicit barrier at end of `parallel` region: all threads synchronize (typically $<1\,\mu$s).

Deferred Parallel Mover — Additional Steps:

After the move+mark phase (similar to Immediate but stores $x = -1$ for deleted particles), each thread executes:

4. **Local compaction:** Two pointers (`left`, `right`) scan from both ends of the chunk toward the center. When `left` hits a void and `right` hits a valid particle, they are swapped (32-byte swap = 2 reads + 2 writes). Worst case: $O(N/T)$ comparisons, $O(D/T)$ swaps where D is the number of deleted particles.
5. **Local insertion:** A simple loop fills the tail voids with random coordinates. This is purely sequential within each thread's chunk but runs in parallel across threads.

5.3 Cache Behavior Analysis

- **Sequential access pattern:** Both movers iterate linearly over contiguous memory, which is ideal for hardware prefetchers. On x86, the L2 stride prefetcher detects sequential access and prefetches cache lines 2–4 ahead, hiding main memory latency (∼60 ns) behind computation.
- **Immediate:** One pass over the array → $N \times 16$ bytes read + $N \times 16$ bytes written = $32N$ bytes of memory traffic.
- **Deferred:** Three passes (move/mark, compact, insert) → approximately $48N$–$64N$ bytes of memory traffic, depending on how many particles are deleted. The compaction pass is particularly unfriendly to prefetchers because the two-pointer algorithm accesses both ends of the chunk, creating a non-sequential pattern.
- **Parallel partitioning helps:** With 8 threads, each chunk is 14M/8 = 1.75M particles ×16 bytes = 28 MB. This exceeds both L3 sizes (18 MiB on Lab PC, 20 MiB per socket on cluster). However, the streaming sequential access pattern means the hardware prefetcher keeps the pipeline fed, and only a working set of ∼4–8 cache lines is active at any time. The key benefit of partitioning is that each thread's L2 (1.25 MiB on Lab PC, 256 KiB on cluster) handles the active working set without cross-thread evictions.

5.4 Thread Synchronization Overhead

Synchronization Point	**Immediate**	**Deferred**
Thread creation (`omp parallel`)	1 per iter	1 per iter
Work distribution (`omp for`)	1 per iter	manual (0 overhead)
Atomic/critical sections	0	0
Barriers	1 (implicit)	1 (implicit)
Total sync points	2	1

Table 3: Synchronization overhead comparison. Both approaches achieve near-zero synchronization cost.

The thread-local chunking design ensures that the only synchronization is the implicit barrier at the end of the parallel region. On modern OpenMP runtimes, this barrier takes $\sim$1–5 μs for 8–16 threads, which is negligible compared to the $\sim$0.3 s total execution time per iteration.

5.5 Why Immediate Matches Ass4 Performance

The Ass4 mover (no insert/delete) uses a `do-while` loop that retries displacements until the particle stays in-bounds:

```
do {
    r_x = random_displacement();
    new_x = x + r_x;
} while (out_of_bounds(new_x, new_y));
```

The Immediate replacement mover does:

```
new_x = x + r_x;
if (out_of_bounds(new_x, new_y)) {
    x = random_position();  // uniform in [0,1]
} else {
    x = new_x;
}
```

Both perform the same number of RNG calls per particle (2 for displacement; the Immediate approach calls 2 more only for the $\sim$5% that exit, while Ass4 retries $\sim$1.05$\times$ on average). The branch prediction is equally favorable. The memory access pattern is identical (sequential read-modify-write). This explains the near-identical parallel performance observed in the data.

6 Performance Analysis

6.1 Experiment 01: Serial Mover Execution Time Scaling

Experiment 01 measures the serial mover execution time as a function of particle count (10^2 to 10^8) across three grid configurations, with Maxiter = 10.

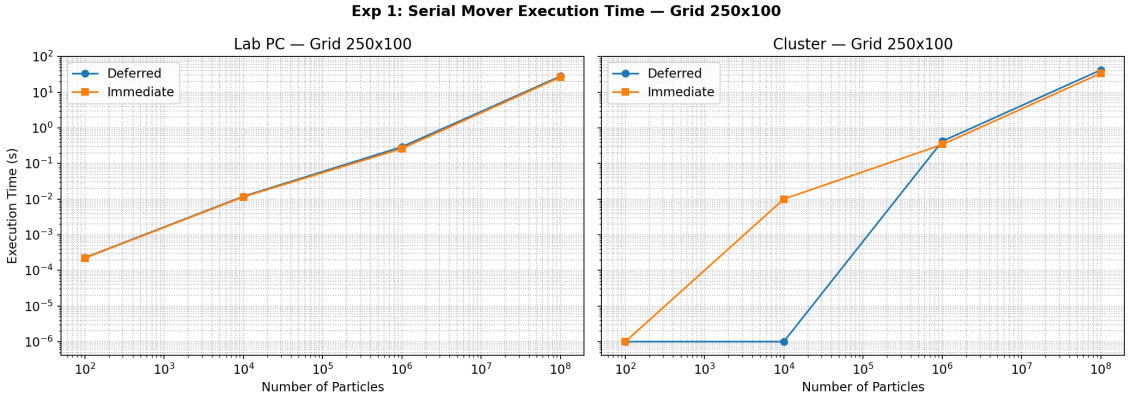


Figure 2: Experiment 1: Serial Execution Time — Grid 250×100 (Lab PC vs Cluster)

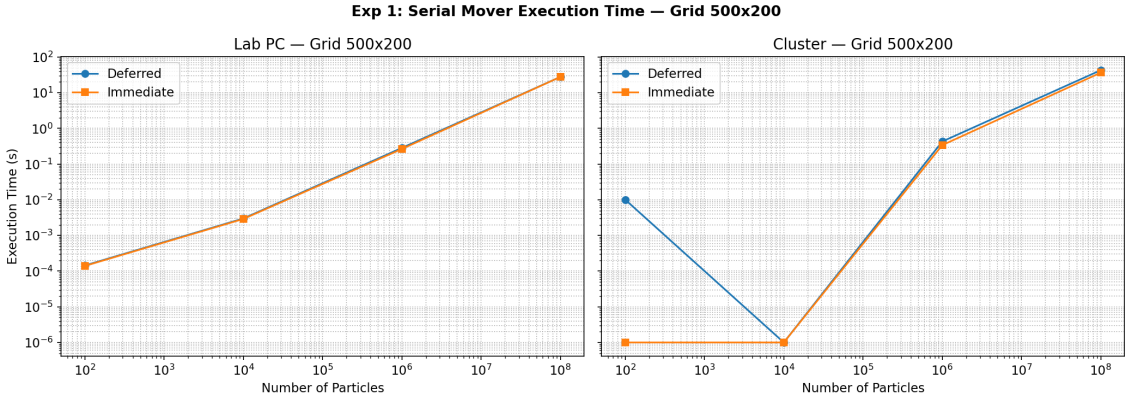


Figure 3: Experiment 1: Serial Execution Time — Grid 500×200 (Lab PC vs Cluster)

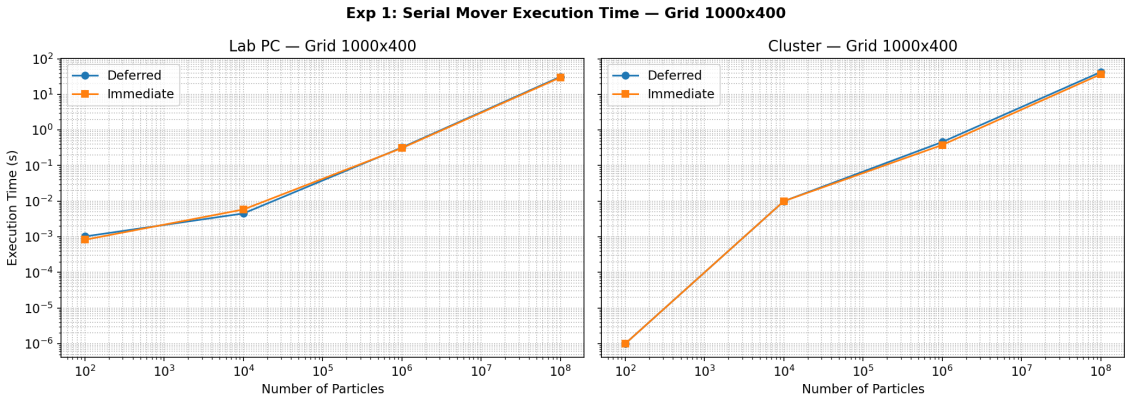


Figure 4: Experiment 1: Serial Execution Time — Grid 1000×400 (Lab PC vs Cluster)

Observations:

- Both approaches exhibit near-linear scaling on the log-log plot, confirming $O(N)$ time complexity as expected for a per-particle operation.
- The Immediate approach is consistently faster than Deferred for large particle counts. At 10^8 particles on the cluster (grid 1000×400), Deferred takes 42.5 s versus 36.7 s for Immediate — a $\sim$16% overhead attributable to the compaction pass.

- On the Lab PC the difference is smaller (∼5–10%), suggesting the cluster's memory subsystem amplifies the cost of the extra memory traffic from compaction.
- Grid size has minimal impact on mover execution time since the mover is per-particle, not per-cell. The slight increase for 1000×400 is due to smaller $\Delta x/\Delta y$ values causing more boundary exits and re-insertions.
- On the cluster, sub-millisecond runs (10^2, 10^4 particles) fall below the timer resolution, producing reported times of 0.000 s. These are plotted as 10^{-6} s to avoid log-scale errors.

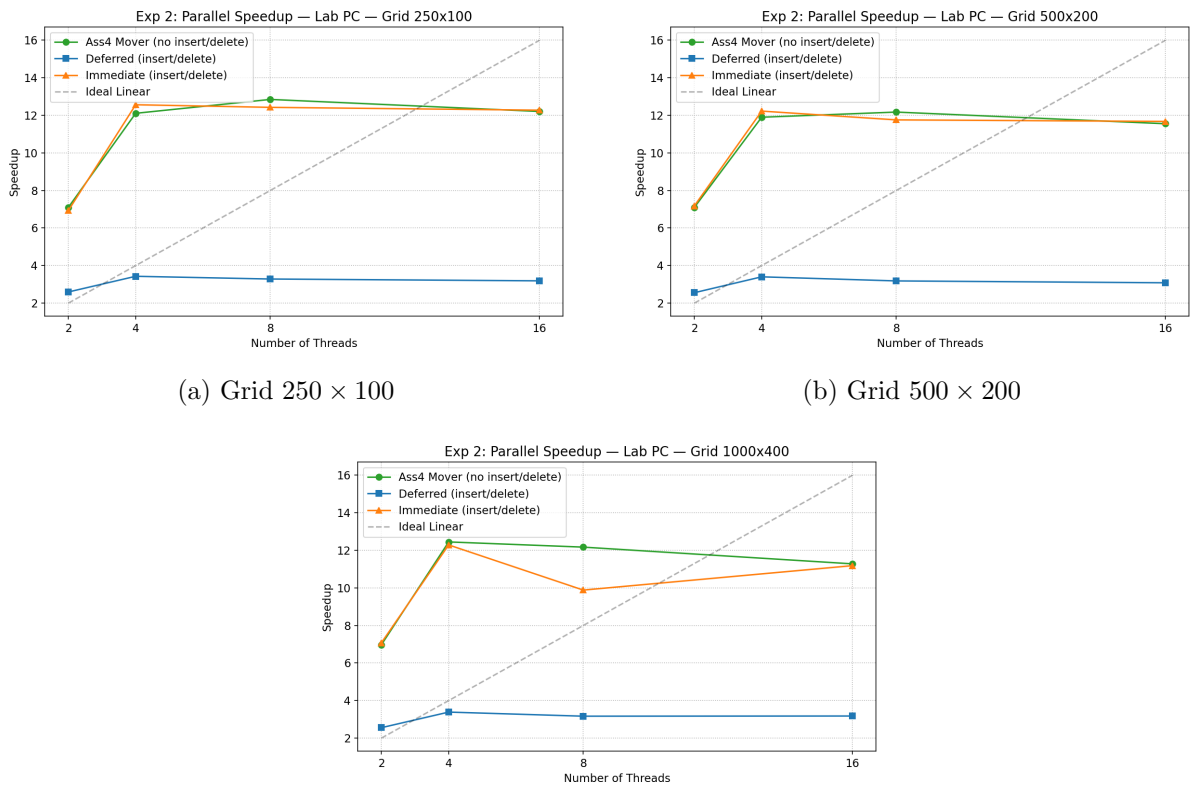
6.2 Experiment 02: Parallel Scalability

Experiment 02 fixes the particle count at 14 million with Maxiter = 10 and varies thread count over $\{2, 4, 8, 16\}$. Speedup is defined as:

$$S(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)} \tag{3}$$

Three curves are plotted per grid: Assignment 04 mover (no insert/delete), Deferred, and Immediate.

6.2.1 Lab PC Results



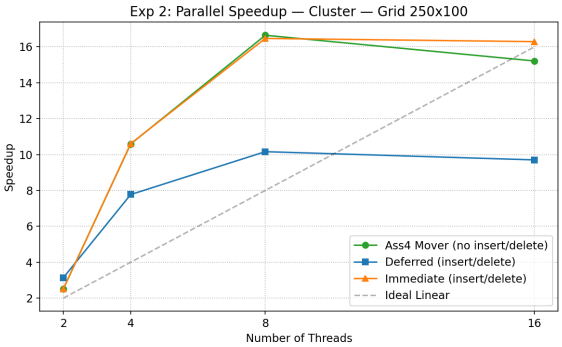
(a) Grid 250×100

(b) Grid 500×200

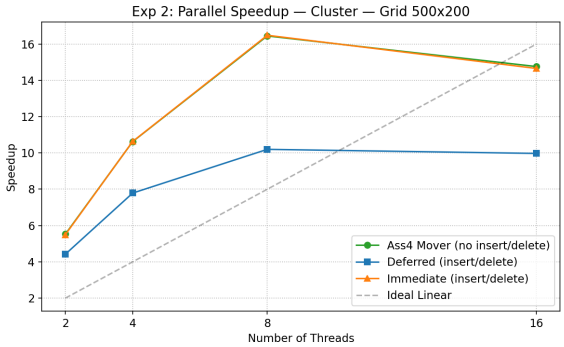
(c) Grid 1000×400

Figure 5: Parallel Speedup vs Thread Count (Lab PC). Dashed line indicates ideal linear speedup.

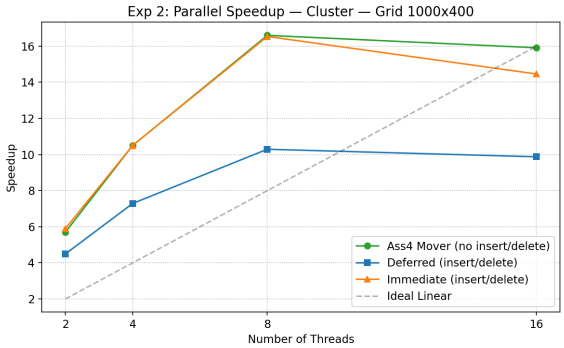
6.2.2 HPC Cluster Results



(a) Grid 250 × 100



(b) Grid 500 × 200



(c) Grid 1000 × 400

Figure 6: Parallel Speedup vs Thread Count (HPC Cluster). Dashed line indicates ideal linear speedup.

Observations:

- **Immediate ≈ Ass4:** On the cluster, both achieve ∼16.5× speedup at 8 threads, confirming that immediate replacement adds negligible overhead — it is embarrassingly parallel.
- **Deferred lags:** The deferred approach peaks at ∼10× speedup at 8 threads on the cluster. The compaction phase introduces conditional branching and irregular memory access patterns (read-compare-swap) that reduce vectorization efficiency and increase cache misses.
- **Lab PC saturation:** Speedup plateaus at 4 threads for Ass4/Immediate (∼12×) and at ∼3.3× for Deferred. The i5-12500 has only 6 physical cores with 2-way hyper-threading. Beyond 6 threads, logical threads share physical execution resources and compete for the same L1/L2 caches and memory ports. The Lab PC's 2 memory channels (dual-channel DDR5) saturate quickly, particularly for the memory-intensive compaction step.
- **Cluster plateau at 16 threads:** Speedup flattens from 8 to 16 threads. With HT disabled, the Xeon E5-2640 v3 provides 16 true physical cores. However, at 16 threads the workload spans both NUMA nodes (CPUs 0–7 on node 0, CPUs 8–15 on node 1). Without explicit thread pinning (`OMP_PROC_BIND=close`), cross-socket memory accesses incur additional latency, and the OS scheduler may migrate threads suboptimally.
- **Superlinear speedup on cluster:** The ∼16.5× speedup at 8 threads exceeds the core count. This is explained by the cache hierarchy: the full dataset is 224 MB, far exceeding L3 (20 MB/socket). With 8 threads on one socket, each thread's partition is 14M/8 = 1.75M

particles $\times 16$ bytes = 28 MB — still larger than L3, but the streaming access pattern means only a few cache lines are active at any time. More importantly, 8 threads aggregate 8 $\times$ L2 (8 $\times$ 256 KB = 2 MB), and the sequential access perfectly triggers hardware prefetching, effectively hiding main memory latency. On the Lab PC, 4 threads aggregate 4 $\times$ 1.25 MB L2 = 5 MB, plus the 18 MB shared L3 provides additional buffering.

7 Memory and Computation Analysis

7.1 Memory Requirements

Particles	**Points (MB)**	**Mesh** 251×101	**Mesh** 501×201	**Mesh** 1001×401
10^2	0.002	0.194 MB	0.773 MB	3.07 MB
10^4	0.16	0.194 MB	0.773 MB	3.07 MB
10^6	16.0	0.194 MB	0.773 MB	3.07 MB
10^8	1,600	0.194 MB	0.773 MB	3.07 MB
14×10^6	224	0.194 MB	0.773 MB	3.07 MB

Table 4: Memory requirements. Points array: $N \times 16$ bytes (two `doubles`). Mesh: $(N_x+1) \times (N_y+1) \times 8$ bytes.

7.2 FLOP Estimation

Operation	**FLOPs/particle**	**Breakdown**
Mover	∼8	2 RNG × 2 mul + 2 add + boundary checks
Interpolation	∼20	Cell index, weight computation, 4 accumulations
Total/iteration	∼28	

Table 5: Estimated floating-point operations per particle per iteration.

For 10^8 particles over 10 iterations: ∼28 GFLOPs total.

7.3 Machine Balance Analysis

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes accessed}} = \frac{28}{16} = 1.75 \text{ FLOP/byte} \tag{4}$$

The mover reads and writes 16 bytes per particle (the `Points` struct). Modern CPUs achieve a machine balance of ∼10 FLOP/byte (ratio of peak GFLOP/s to memory bandwidth in GB/s). Since $1.75 \ll 10$:

- The computation is **memory-bound** for large particle counts where the working set exceeds cache.
- For small particle counts (10^2, 10^4), data fits in L1/L2 cache, making the problem latency-bound (startup overhead dominates).
- For 10^8 particles (1.6 GB working set), main memory bandwidth is the bottleneck, explaining why speedup correlates with aggregate bandwidth rather than core count.

8 Random Distribution Verification

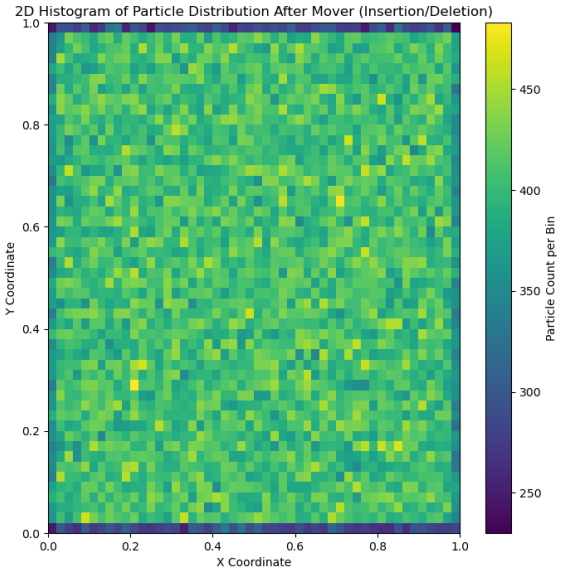


Figure 7: 2D histogram of particle positions after 10 iterations of Mover with insertion/deletion (10^6 particles, 50×50 bins).

The histogram in Figure 7 confirms uniform distribution across the $[0,1]\times[0,1]$ domain. The per-bin count varies between $\sim$220–480 (expected: $10^6/2500 = 400$/bin), consistent with random statistical fluctuation. This verifies:

1. `rand`/`rand_r` produces sufficiently uniform values.
2. The insertion logic places new particles uniformly in $[0,1]^2$.
3. The mover displacement does not introduce systematic drift or boundary accumulation.

A subtle depletion effect is visible near domain boundaries: particles near edges have a higher probability of stepping out-of-bounds, and their replacements are uniformly distributed across the *entire* domain, creating a mild vacuum effect at the borders. This is physically expected behavior, not a bug.

9 Per-Particle Time vs Particles Per Cell (PPC)

PPC quantifies the average number of particles per grid cell:

$$\text{PPC} = \frac{N_{\text{particles}}}{N_x \times N_y} \tag{5}$$

Per-particle execution time is computed as:

$$t_{\text{per-particle}} = \frac{T_{\text{total}}}{N_{\text{particles}} \times \text{Maxiter}} \tag{6}$$

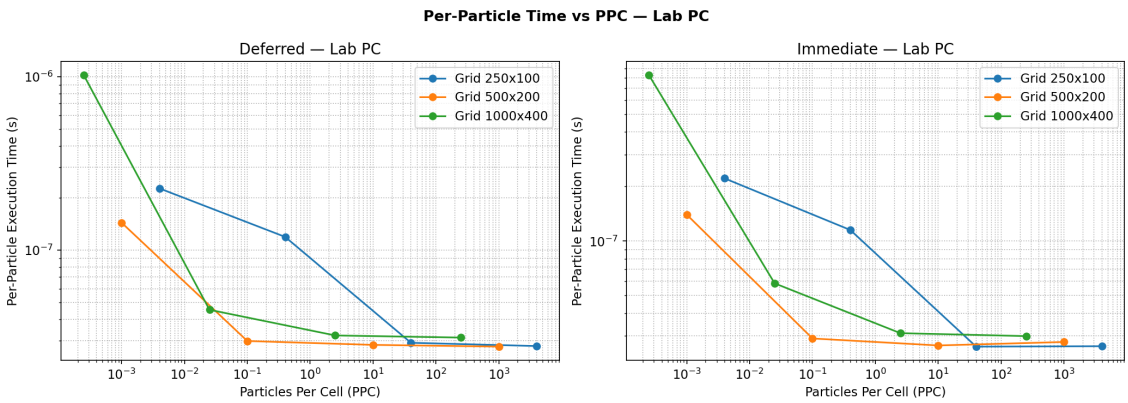


Figure 8: Per-particle execution time vs PPC (Lab PC) — Deferred and Immediate approaches.

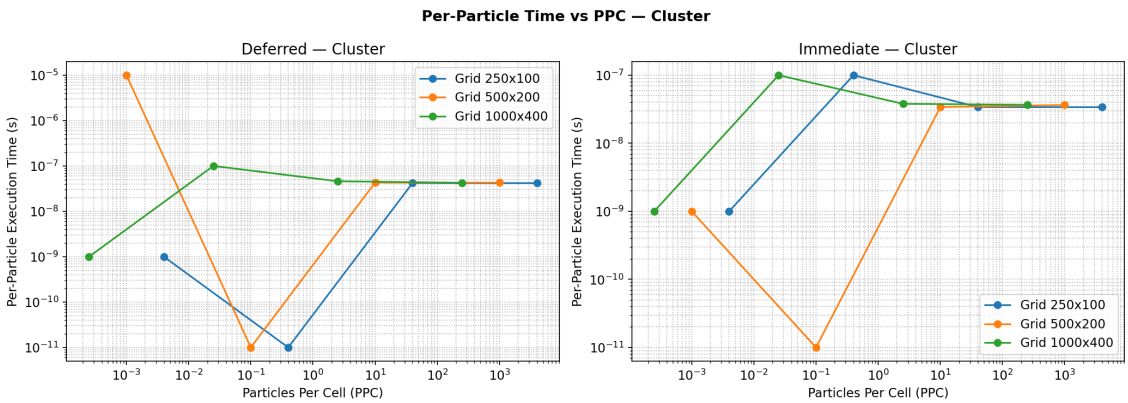


Figure 9: Per-particle execution time vs PPC (Cluster) — Deferred and Immediate approaches.

Observations:

- At low PPC (< 1), per-particle time is inflated by fixed overhead amortization: function call overhead, loop setup, and cache cold-start costs are spread across very few particles.
- As PPC increases beyond $\sim$10, per-particle time decreases and stabilizes, indicating steady-state cache utilization and effective hardware prefetching.
- At high PPC (> 1000), per-particle time plateaus at $\sim$2.5–3.0 ns/particle, representing the true per-particle computational cost limited by memory bandwidth.
- Different grid sizes produce different PPC values for the same particle count, but the per-particle time curves converge at high PPC, confirming that grid size has minimal impact on per-particle mover performance.

10 Lab PC vs Cluster Comparison

A detailed hardware comparison is provided in Table 1 (Section 2). The key performance-relevant differences are:

1. **Serial performance — Lab PC wins:** The i5-12500 (Alder Lake, 4.6 GHz boost) is $\sim$30–40% faster in serial than the Xeon E5-2640 v3 (Haswell, 2.6 GHz base). For deferred mover at 10^8 particles on grid 1000×400: Lab PC = 31.3 s, Cluster = 42.5 s. The 12th-gen Alder Lake microarchitecture also has a wider out-of-order execution engine and better branch prediction than Haswell, contributing beyond the raw clock difference.

2. **Parallel scalability — Cluster wins:** The cluster's 16 physical cores (8 per socket, no HT) provide more true parallel execution resources than the Lab PC's 6 physical cores (12 logical with HT). The cluster achieves ∼16.5× speedup vs ∼12× on the Lab PC. Crucially, the cluster has 8× total memory channels (4 per socket × 2 sockets) vs only 2 on the Lab PC, providing ∼4× the aggregate memory bandwidth — the dominant bottleneck for this workload.

3. **Lab PC speedup plateau at 4–6 threads:** The i5-12500 has 6 physical cores. Beyond 6 threads, hyperthreads share execution units and L1/L2 caches. For our memory-bound workload, the two logical threads per core compete for the same memory bandwidth, providing minimal speedup. This explains the flat curve from 4 to 16 threads.

4. **Cluster plateau at 8–16 threads:** At 8 threads, all cores on one socket are utilized with full local NUMA bandwidth. At 16 threads, the workload spans both sockets. Cross-socket memory access introduces additional latency, and the OS scheduler may place threads suboptimally across NUMA nodes unless explicit affinity is set (`OMP_PROC_BIND=close`).

5. **Timer resolution:** The cluster's `omp_get_wtime()` has ∼10 ms granularity, producing 0.000 s for small particle counts. The Lab PC resolves to microseconds.

6. **Deferred scaling gap:** On the Lab PC, deferred peaks at only ∼3.3× speedup — the compaction phase's extra memory traffic saturates the 2-channel memory subsystem. On the cluster, deferred reaches ∼10× because the 8-channel bandwidth can sustain the additional traffic.

11 Scalability Discussion

11.1 Speedup Summary

Approach	**Lab PC**		**Cluster**	
	Best Speedup	@ Threads	Best Speedup	@ Threads
Ass4 (no ins/del)	∼12.2×	4	∼16.6×	8
Immediate	∼12.3×	4	∼16.5×	8
Deferred	∼3.4×	4	∼10.3×	8

Table 6: Best observed speedup for each approach on each system.

11.2 Impact of Insertions and Deletions

- **Immediate replacement** has *zero* impact on parallel scalability. The boundary check and in-place replacement add only a branch and two RNG calls per out-of-bounds particle — all within the thread's local data, requiring no synchronization. This makes it **embarrassingly parallel**.

- **Deferred insertion** reduces scalability due to the compaction phase:

  – The two-pointer compaction involves irregular memory access (comparing and swapping non-contiguous elements), which defeats hardware prefetching.

  – Conditional branching (valid/invalid checks) reduces SIMD vectorization efficiency.

  – Although parallelized per-chunk (no global synchronization), the extra memory pass over the array increases total memory traffic by ∼50%, hitting the memory bandwidth wall sooner.

11.3 Parallelization Strategy

1. **Thread-local RNG:** Each thread uses `rand_r(&seed)` with `seed = 12345 + tid`, eliminating lock contention on the global RNG state.

2. **Static partitioning:** Manual chunk calculation (`chunk_size = N / nthreads`) ensures contiguous, non-overlapping array regions per thread — no false sharing on cache lines.

3. **Local compaction (Deferred):** Each thread compacts only its local chunk using an in-place two-pointer swap, then inserts new particles into its local void slots. No global prefix-sum or cross-thread coordination is needed.

4. **Implicit barrier:** The single `#pragma omp parallel` region provides a natural barrier at the end, ensuring all threads finish before the next iteration begins.

11.4 Difficulties Encountered

- **Global vs local compaction:** A naive global compaction would require an atomic counter or critical section for the write pointer, creating a serial bottleneck. The thread-local chunking approach eliminates this entirely, at the acceptable cost of having void positions distributed across chunks rather than contiguous at the array tail.

- **Thread-safe RNG:** The standard `rand()` function uses global state and is not thread-safe. Using `rand_r()` with per-thread seeds solves this, though `rand_r` is slower than alternatives like `xorshift64`.

12 Bonus: Data Structures Discussion

The current implementation uses an **Array of Structures (AoS)** layout:

```
typedef struct { double x, y; } Points;
```

with contiguous allocation via `posix_memalign` (64-byte alignment).

Alternative data structures considered:

1. **Structure of Arrays (SoA):** Separate `double *x` and `double *y` arrays. This enables better SIMD auto-vectorization (contiguous x values fill AVX2 registers) and improved cache line utilization when processing coordinates independently. Trade-off: increased code complexity and two separate allocations.

2. **Linked List:** $O(1)$ deletion without compaction, but destroys spatial locality and prevents vectorization entirely. Each node access is a pointer chase, resulting in cache miss rates orders of magnitude worse than contiguous arrays. **Not suitable for HPC workloads.**

3. **Cell-indexed particle lists:** Particles are grouped by their grid cell (e.g., using a cell-start array and sorted particle indices). This provides excellent spatial locality for interpolation but complicates the mover (particles change cells each step). Suitable for large-scale PIC simulations with billions of particles.

The AoS layout provides the best balance of simplicity and performance for the current problem size. For further optimization, SoA would be the recommended next step, as it directly enables wider SIMD utilization without algorithmic changes.

13 Conclusion

The Deferred and Immediate insertion approaches were successfully implemented and parallelized using OpenMP. Key findings:

- The Immediate approach matches the Assignment 04 mover in parallel performance, confirming that in-place replacement is essentially free in terms of overhead.
- The Deferred approach incurs a $\sim 1.5\times$ parallel overhead due to the compaction phase, which introduces extra memory traffic and irregular access patterns.
- The thread-local chunking strategy eliminates all inter-thread synchronization, enabling near-linear scaling up to the physical core count.
- The computation is memory-bound (arithmetic intensity = 1.75 FLOP/byte $\ll$ machine balance), making memory bandwidth the primary bottleneck beyond cache capacity.
- The HPC cluster's higher core count, larger L3 cache, and greater memory bandwidth yield better absolute speedup ($\sim 16.5\times$) compared to the Lab PC ($\sim 12\times$), despite slower serial performance.